

SFTL Debugging Plan

Focused reproduction and failure analysis for temporal-shift CTR experiments

Context. Current experiments show that Han ARW adapts clearly under abrupt synthetic drift, AdaMoE adapts moderately, Differentiable Forgetting reacts but recovers slowly, while SFTL is substantially worse across all four synthetic modes and especially poor under abrupt drift. On real Criteo data, all methods are close because the natural drift is shallow.

Objective

Determine whether SFTL's failure is caused by an implementation error, a timescale/hyperparameter mismatch, or a genuine mismatch between the method's ranking-oriented trajectory objective and calibrated CTR prediction.

Debugging principle

- **Do not tune everything at once.** Establish implementation invariants first.
- Separate **ranking quality** from **probability calibration**. SFTL may preserve AUC while destroying log loss.
- Instrument the optimization. Track the BCE term, trajectory terms, margins, logits, and gradient magnitudes.
- Give SFTL one bounded debugging pass. If the faithful implementation still fails, treat that as a result rather than an endless tuning problem.

Primary failure hypotheses

Hypothesis	Observable symptom	Test
H1. Implementation bug	SFTL with trajectory losses disabled differs from ordinary sequential training.	Set $\lambda_s = \lambda_r = 0$ and compare to the baseline.
H2. Calibration / confidence runaway	AUC remains reasonable but log loss, Brier score, or logit magnitude deteriorates.	Track AUC, log loss, calibration and margins for working/slow/fast learners.
H3. EMA timescale mismatch	Fast learner behaves too slowly or too noisily for the synthetic domain length.	Sweep interpretable EMA half-lives.
H4. Trajectory term dominates BCE	Trajectory-gradient norm becomes comparable to or larger than the BCE gradient.	Measure gradient ratios and tune λ by contribution, not arbitrary scale.
H5. Genuine method limitation	Faithful implementation works on native-style reproduction but fails on our drift benchmark.	Run one lightweight reproduction sanity check.

Stage 1 - Establish implementation invariants

Run the smallest possible faithful SFTL configuration before any hyperparameter search.

Required checks

- Set the trajectory-loss coefficients to zero: $\lambda_s = \lambda_f = 0$.
- Verify that the working learner matches the ordinary sequential/incremental baseline to numerical tolerance.
- Verify slow-learner update timing: copy working weights only at the intended temporal-domain boundary.
- Verify fast-learner update timing: exponential moving average after the intended mini-batch updates.
- Stop gradients through the slow and fast trajectory learners if required by the implementation.
- Verify which learner is used for inference. Report working, slow, and fast predictions separately during debugging.

Gate: If the zero-trajectory configuration does not reproduce the ordinary baseline, stop. Fix the implementation before doing any tuning.

Stage 2 - Separate ranking from calibration

Record metrics for **working**, **slow**, and **fast** learners separately on every evaluation day.

Metric family	Record
Prediction	AUC, log loss, Brier score, PR-AUC
Calibration	mean predicted CTR, calibration error / reliability bins
Confidence	mean logit , logit std., max logit
Optimization	BCE loss, slow trajectory loss, fast trajectory loss
Margins	positive-negative margin for working, slow and fast learners

Interpretation

- **AUC stable, log loss explodes:** likely ranking/calibration mismatch or runaway confidence.
- **AUC and log loss both collapse:** likely optimization, EMA, or implementation failure.
- **Fast learner bad but working learner healthy:** investigate EMA/inference rather than the whole method.

Stage 3 - Test the runaway-margin hypothesis

Track the trajectory margins throughout training, especially before and immediately after the synthetic shift.

For each mini-batch or logging interval, save:

- working positive-negative margin u
- fast-teacher margin v_f
- slow-teacher margin v_s
- differences $u-v_f$ and $u-v_s$
- mean and maximum absolute logits

A convincing runaway signature is monotone confidence growth before any distribution shift, followed by a sharp log-loss penalty once the regime changes.

Stage 4 - Sweep EMA memory using half-life

Parameterize EMA memory by an interpretable half-life H in optimizer steps rather than reasoning directly about α .

Use the convention:

$$\alpha = 2^{-1/H}$$

Test a compact grid such as $H \in \{10, 100, 1000, 10000\}$ updates, adjusted to the actual number of updates per synthetic day.

- Very small H : fast learner nearly tracks the working learner.
- Very large H : fast learner behaves as long memory.
- Plot effective half-life against post-shift recovery time and pre-shift calibration.

Important: If the paper's prose and equation imply different EMA conventions, test the alternative convention only as a diagnostic and document which one is considered faithful.

Stage 5 - Ablate the trajectory signals

Run exactly four variants under the same seeds and optimizer settings:

Variant	Loss
A. BCE only	L_{CE}
B. Slow only	$L_{CE} + \lambda_s L_{tra}^s$
C. Fast only	$L_{CE} + \lambda_f L_{tra}^f$
D. Full SFTL	$L_{CE} + \lambda_s L_{tra}^s + \lambda_f L_{tra}^f$

- Evaluate each variant with the working learner and fast learner at inference.
- If one trajectory signal causes most instability, do not average the diagnosis across the full method.

Stage 6 - Tune trajectory strength by gradient contribution

Avoid an arbitrary λ sweep as the primary diagnostic. Measure how much each trajectory term contributes to the parameter update.

Log the ratios:

$$r_s = \lambda_s \ ||\nabla L_{tra}^s|| / ||\nabla L_{CE}|| \text{ and } r_f = \lambda_f \ ||\nabla L_{tra}^f|| / ||\nabla L_{CE}||$$

Choose λ values that target approximate initial contribution levels such as 1%, 5%, 10%, 25%, and 50% of the BCE gradient norm.

- Also sweep the trajectory warm-up/start point if the implementation exposes it.
- Prefer the smallest trajectory contribution that produces a reproducible AUC or adaptation benefit without harming calibration.
- Use identical seeds across settings so abrupt-drift recovery curves are directly paired.

Stage 7 - Only then test calibration stabilizers

If faithful SFTL improves ranking but damages probability quality, keep the literature implementation unchanged as **SFTL** and test modified versions under separate names.

- Temporal-validation temperature scaling.
- Logit-norm penalty.
- Trajectory-gradient clipping or adaptive balancing.
- Optional confidence cap only as an engineering diagnostic, not as the primary research result.

Do not silently change SFTL and continue labeling it as the original baseline.

Stage 8 - Lightweight native-setting reproduction

If the method remains suspicious after the previous stages, run one limited reproduction close to the paper's intended CTR setting. The goal is qualitative verification, not a full reproduction campaign.

- Use a chronological CTR dataset similar to the published setup.
- Use a reasonably comparable CTR backbone if practical.
- Check whether SFTL produces the expected AUC improvement.
- If it works there but fails on our synthetic benchmark, classify the failure as a domain/shift limitation rather than an implementation failure.

Recommended run matrix

Priority	Run	Purpose
P0	BCE-only SFTL vs ordinary baseline	Implementation invariant
P0	Working / slow / fast metric logging	Locate branch failure
P0	Margin + logit traces	Test confidence escalation
P1	EMA half-life sweep	Check timescale mismatch
P1	Slow-only / fast-only / full ablation	Identify harmful trajectory term
P1	Gradient-ratio λ sweep	Check trajectory dominance
P2	Calibration-stabilized variants	Only if AUC good but log loss bad
P2	Native-setting sanity reproduction	Distinguish method limitation from implementation issue

Decision rules

- **Implementation bug:** BCE-only SFTL does not reproduce the ordinary sequential baseline.
- **Ranking/calibration mismatch:** AUC improves or remains competitive while log loss/Brier deteriorate sharply.
- **Timescale mismatch:** performance changes materially with EMA half-life and one setting restores adaptation.
- **Trajectory instability:** margins/logits grow steadily and trajectory-gradient contribution dominates BCE.
- **Genuine limitation:** a faithful implementation behaves reasonably in a native-style setting but still loses on the controlled temporal-shift benchmark.

Stop criterion

Give SFTL one focused debugging pass through P0 and P1. If the faithful version remains worse than simple rolling-window, Han ARW, and AdaMoE baselines, stop investing in it. Preserve the failure analysis as evidence that trajectory-based slow/fast regularization is not robust for this benchmark.

Final deliverables

- One table with all ablations and primary metrics.
- One figure with daily AUC and log loss around the abrupt shift.
- One figure with logit/margin trajectories.
- One figure or table showing EMA half-life versus recovery time.
- A short written diagnosis selecting one of: bug, calibration mismatch, timescale mismatch, trajectory instability, or genuine limitation.